бюджетное профессиональное образовательное учреждение Вологодской области
«Череповецкий лесомеханический техникум им. В.П. Чкалова»

Специальность **09.02.07** «Информационные системы и программирование»

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

ПП по ПМ.06 Сопровождение информационных систем

Выполнил студент __курса группы ИС- _____

_____

подпись _____

место практики

_____

ООО «Малленом Системс»

Период прохождения:

с « ___» _____ 2026 г.

по «___»_____ 2026 г.

Руководитель практики от предприятия

должность_____

_____

подпись_____

МП

Руководитель практики от техникума: Материкова А.А.

_____

Оценка:_____

«___» _____2026 года

г.Череповец
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 2
1 ОБЩАЯ ХАРАКТЕРИСТИКА ПРЕДПРИЯТИЯ (ОРГАНИЗАЦИИ) 3
1.1 Общая информация об организаци 3
1.2 Роль информационных систем (ИС) в работе организации 3
1.3 Основные используемые технологии в организации 4
2 СОПРОВОЖДЕНИЕ ИНФОРМАЦИОННЫХ СИСТЕМ 5
2.1 Цели и задачи сопровождения ИС 5
2.2 Основные задачи сопровождения ИС 6
2.3 Виды сопровождения информационных систем 7
2.4 Процессы и этапы сопровождения ИС 8
2.5 Методы и инструменты сопровождения 9
2.6 Управление рисками и безопасностью 10
3 ВЫПОЛНЯЕМЫЕ ЗАДАНИЯ 12
ЗАКЛЮЧЕНИЕ 12
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧЧНИКОВ 15
ПРИЛОЖЕНИЯ 17

ВВЕДЕНИЕ

Цель производственной практики: закрепление и углубление теоретических знаний, полученных в процессе обучения, формирование профессиональных компетенций в области сопровождения информационных систем, приобретение практических навыков инсталляции, настройки и технической поддержки программного обеспечения.

Задачи производственной практики:

Изучить информационные системы, используемые на предприятии

Проанализировать процессы сопровождения информационных систем

Выявить типовые ошибки, возникающие в процессе эксплуатации, и определить способы их устранения

Разработать обучающие материалы для пользователей по эксплуатации информационной системы

Применить полученные знания при разработке Web-API приложения

Краткое описание организации

Место прохождения производственной практики — ООО «Малленом Системс». Компания специализируется на разработке и сопровождении программного обеспечения для автоматизации бизнес-процессов. Организация осуществляет полный цикл создания информационных систем: от анализа требований до внедрения и последующей технической поддержки.

Сроки прохождения производственной практики: с 02.03.26 по 22.03.26

Место прохождения: ООО «Малленом Системс», г. Череповец

1 ОБЩАЯ ХАРАКТЕРИСТИКА ПРЕДПРИЯТИЯ (ОРГАНИЗАЦИИ)

1.1 Общая информация об организаци

ООО «Малленом Системс» — динамично развивающаяся компания в сфере информационных технологий, предоставляющая услуги по разработке, внедрению и сопровождению программного обеспечения для предприятий различных отраслей.

Сфера деятельности:

Разработка корпоративных информационных систем

Сопровождение и техническая поддержка программного обеспечения

Интеграция различных информационных систем

Консалтинг в области автоматизации бизнес-процессов

Организационная структура предприятия:

Численность сотрудников: более 100 человек

Руководство компании:

Генеральный директор: Живиця Анна Эдуардовна

Учредители: 10 физических лиц

1.2 Роль информационных систем (ИС) в работе организации

Информационные системы играют ключевую роль в деятельности ООО «Малленом Системс», обеспечивая:

Управление проектами и задачами — координация работы сотрудников, контроль сроков и ресурсов

Взаимодействие с клиентами — ведение базы клиентов, управление продажами и поддержкой

Документооборот — автоматизация процессов согласования и хранения документов

Разработка и сопровождение ПО — управление исходным кодом, отслеживание ошибок

Учет рабочего времени — контроль загрузки сотрудников и планирование ресурсов

В организации используются следующие информационные системы:

Bitrix24 — для управления бизнес-процессами и взаимодействия с клиентами

Yandex Tracker — для управления задачами и проектами

Jira — для управления разработкой и отслеживания ошибок

Git — для контроля версий исходного кода

1.3 Основные используемые технологии в организации

В процессе деятельности предприятие использует современный технологический стек, обеспечивающий эффективную разработку и сопровождение информационных систем.

Категория Используемые технологии Назначение

2 СОПРОВОЖДЕНИЕ ИНФОРМАЦИОННЫХ СИСТЕМ

2.1 Цели и задачи сопровождения ИС

Сопровождение информационных систем (ИС) представляет собой комплекс мероприятий, направленных на обеспечение бесперебойной, эффективной и безопасной эксплуатации программного обеспечения после его внедрения в промышленную эксплуатацию.

Основные цели сопровождения ИС:

Поддержание работоспособности системы на требуемом уровне доступности

Обеспечение соответствия системы изменяющимся потребностям бизнеса

Минимизация времени простоя и оперативное устранение сбоев

Оптимизация производительности и эффективности работы системы

Обеспечение информационной безопасности и целостности данных

Отличие сопровождения от разработки:

Характеристика Разработка Сопровождение

Основная задача Создание нового функционала Поддержка существующего функционала

Характер работ Проектный, инновационный Оперативный, сервисный

Результат Новый программный продукт Стабильная работа системы

Команда Разработчики, аналитики Инженеры поддержки, администраторы

Временной горизонт Фиксированный (проект) Постоянный (цикл эксплуатации)

2.2 Основные задачи сопровождения ИС

Поддержание работоспособности системы:

Круглосуточный мониторинг состояния серверов и приложений

Оперативное реагирование на инциденты и сбои

Обеспечение доступности системы для пользователей (SLA)

Управление нагрузкой и масштабирование при необходимости

Устранение ошибок и оптимизация производительности:

Анализ логов и выявление причин сбоев

Исправление ошибок в программном коде

Оптимизация запросов к базе данных

Настройка параметров системы для повышения производительности

Устранение утечек памяти и других ресурсных проблем

Адаптация системы к изменяющимся требованиям:

Обновление нормативно-справочной информации

Настройка прав доступа при изменении штатного расписания

Интеграция с новыми внешними системами

Адаптация к изменениям законодательства

Обновление и модернизация компонентов:

Установка обновлений безопасности и исправлений

Обновление версий библиотек и фреймворков

Миграция данных при переходе на новые версии

Замена устаревшего оборудования и программного обеспечения

2.3 Виды сопровождения информационных систем

Корректирующее сопровождение (исправление сбоев):

Выявление и устранение ошибок в программном коде

Исправление проблем, возникших в процессе эксплуатации

Оперативное реагирование на инциденты

Устранение последствий сбоев и восстановление данных

Адаптивное сопровождение (подстройка под новые условия):

Адаптация к изменениям операционной системы и аппаратного обеспечения

Настройка под новое программное окружение

Изменение параметров конфигурации при изменении бизнес-процессов

Обеспечение совместимости с новыми версиями смежных систем

Совершенствующее сопровождение (добавление функций):

Реализация новых требований пользователей

Улучшение существующего функционала

Расширение возможностей системы

Оптимизация пользовательского интерфейса

Профилактическое обслуживание (предупреждение проблем):

Регулярное резервное копирование данных

Мониторинг и анализ состояния системы

Плановое обновление компонентов

Проверка журналов событий и выявление потенциальных проблем

2.4 Процессы и этапы сопровождения ИС

Процесс сопровождения информационных систем строится на основе циклической модели, включающей следующие этапы:

1. Прием и регистрация обращений

Фиксация заявки в системе учета обращений (Service Desk)

Определение категории и приоритета проблемы

Назначение ответственного специалиста

2. Анализ и диагностика

Воспроизведение ситуации для выявления причин

Анализ логов и журналов событий

Оценка сложности и сроков решения

При необходимости — эскалация на вышестоящий уровень поддержки

3. Разработка решения

Исправление ошибки или настройка параметров

Разработка обновления или патча

Тестирование предложенного решения

Документирование выполненных изменений

4. Согласование и внедрение

Развертывание решения на тестовом стенде

Согласование с пользователем или заказчиком

Плановое внедрение в продуктивную среду

Мониторинг после внедрения

5. Закрытие и анализ

Проверка результата и подтверждение решения проблемы

Информирование пользователя

Обновление базы знаний

Анализ для предотвращения подобных проблем в будущем

2.5 Методы и инструменты сопровождения

Инструменты сопровождения, используемые в организации:

Назначение	Инструменты	Применение
Учет обращений	Jira Service Management, Yandex Tracker	Регистрация и отслеживание заявок
Мониторинг	Zabbix, Prometheus, Grafana	Контроль состояния систем
Логирование	ELK Stack (Elasticsearch, Logstash, Kibana)	Сбор и анализ журналов
Управление версиями	Git, GitHub, GitLab	Контроль изменений кода
Удаленное администрирование	SSH, RDP, TeamViewer	Доступ к серверам и рабочим станциям

Тестирование Postman, Swagger, pytest Проверка работоспособности

Документирование Confluence, Wiki Ведение базы знаний

Методы сопровождения:

Активный мониторинг — постоянный контроль ключевых показателей

Проактивное выявление проблем — анализ трендов и предиктивная диагностика

Автоматизация рутинных операций — скрипты для типовых задач

Регламентированное обновление — плановые окна обслуживания

2.6 Управление рисками и безопасностью

Идентификация рисков при сопровождении:

Категория риска Примеры Вероятность Влияние

Технические риски Потеря данных при обновлении Средняя Высокое

Несовместимость обновлений Низкая Высокое

Отказ оборудования Средняя Высокое

Операционные риски Длительный простой системы Низкая Высокое

Человеческая ошибка Средняя Среднее

Безопасность Несанкционированный доступ Низкая Критическое

Утечка конфиденциальных данных Низкая Критическое

Обеспечение информационной безопасности:

Разграничение прав доступа по принципу минимальных привилегий

Использование защищенных протоколов для удаленного доступа (SSH, HTTPS)

Регулярный аудит действий администраторов и пользователей

Шифрование конфиденциальных данных при хранении и передаче

Двухфакторная аутентификация для критических систем

Резервное копирование и восстановление данных:

Вид резервирования Периодичность Срок хранения Назначение

Полное копирование Ежедневно 30 дней Восстановление всей системы

Инкрементное копирование Каждый час 7 дней Восстановление последних изменений

Копирование транзакций В реальном времени 24 часа Точка восстановления

Архивное копирование Ежемесячно 1 год Долгосрочное хранение

Политика резервного копирования:

Хранение копий на различных носителях (локальные диски, облачное хранилище)

Регулярная проверка восстановления из резервных копий

Документирование процедур восстановления

Разделение прав на создание и восстановление копий

3 ВЫПОЛНЯЕМЫЕ ЗАДАНИЯ

ЗАКЛЮЧЕНИЕ

Итоги практики: достижение целей, приобретенные навыки

В ходе прохождения производственной практики в ООО «Малленом Системс» были успешно выполнены все поставленные задачи:

Изучена деятельность предприятия:

Ознакомлен с организационной структурой и режимом работы

Изучены должностные инструкции ИТ-специалистов

Проанализирована роль информационных систем в работе организации

Освоены процессы сопровождения информационных систем:

Изучены цели, задачи и виды сопровождения

Рассмотрены этапы процесса сопровождения

Изучены методы и инструменты сопровождения

Проанализированы риски и способы их минимизации

Изучены информационные системы предприятия:

Bitrix24 — система управления бизнесом

Yandex Tracker — система управления задачами

Jira — система управления разработкой ПО

Выполнены практические задания:

Проведен анализ типовых ошибок при эксплуатации ИС

Разработаны способы их устранения

Созданы обучающие материалы для пользователе

Разработано Web-API приложение:

Реализованы 4 основных метода API

Выполнена интеграция с базой данных SQLite

Создана интерактивная документация Swagger UI

Проведено тестирование и отладка приложения

Приобретенные профессиональные навыки:

Настройка и сопровождение информационных систе

Разработка Web-API приложений на Python с использованием FastAPI

Работа с системами контроля версий Git

Разработка технической документации и обучающих материалов

Анализ и устранение ошибок при эксплуатации ПО

Применение методов резервного копирования и восстановления данных

Значение полученного опыта для профессионального роста

Полученный в ходе практики опыт имеет важное значение для дальнейшего профессионального развития. Освоенные навыки сопровождения информационных систем, разработки Web-API приложений и работы с современными технологиями (Python, FastAPI, SQLite, Git) являются востребованными на рынке труда.

Практика позволила:

Применить теоретические знания в реальных производственных условиях

Познакомиться с корпоративной культурой и организацией работы ИТ-специалистов

Развить навыки самостоятельной работы и решения профессиональных задач

Сформировать понимание процессов сопровождения информационных систем

Полученные компетенции станут основой для дальнейшего обучения и успешной трудовой деятельности в сфере информационных технологий.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧЧНИКОВ

1. ГОСТ 7.32-2001 «Отчет о научно-исследовательской работе. Структура и правила оформления» [Электронный ресурс]/ Электронный правовой и нормативно-технической документации- http://docs.cntd.ru/document/gost-7-32-2001-sibid

2. Официальная документация Python 3.14.3 [Электронный ресурс]/ Электронный правовой и нормативно-технической документации — https://docs.python.org/3/

3. Официальная документация FastAPI [Электронный ресурс]/ Электронный правовой и нормативно-технической документации — https://fastapi.tiangolo.com/

4. Документация библиотеки Pillow [Электронный ресурс]/ Электронный правовой и нормативно-технической документации — https://pillow.readthedocs.io/

5. Документация SQLAlchemy [Электронный ресурс]/ Электронный правовой и нормативно-технической документации — https://www.sqlalchemy.org/

6. Стандарт PEP 8 — Style Guide for Python Code [Электронный ресурс]/ Электронный правовой и нормативно-технической документации — https://peps.python.org/pep-0008/

7. Официальная документация Git [Электронный ресурс]/ Электронный правовой и нормативно-технической документации — https://git-scm.com/

8. Официальная документация Bitrix24 [Электронный ресурс]/ Электронный правовой и нормативно-технической документации — https://www.bitrix24.ru/features/

9. Официальная документация Yandex Tracker [Электронный ресурс]/ Электронный правовой и нормативно-технической документации — https://yandex.ru/support/tracker/

10. Официальная документация Jira [Электронный ресурс]/ Электронный правовой и нормативно-технической документации — https://www.atlassian.com/software/jira

ПРИЛОЖЕНИЯ

```python
from fastapi import FastAPI, UploadFile, File, HTTPException, Form, Depends
from sqlalchemy import create_engine, Column, Integer, String, DateTime
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session
from PIL import Image
import datetime
import os

app = FastAPI()

UPLOAD_FOLDER = "uploads"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

SQLALCHEMY_DATABASE_URL = "sqlite:///./images.db"
engine = create_engine(SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

class ImageModel(Base):
    __tablename__ = "images"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String)
    file_path = Column(String)
    size_bytes = Column(Integer)
    width = Column(Integer)
    height = Column(Integer)
    file_type = Column(String)
    created_at = Column(DateTime, default=datetime.datetime.utcnow)

Base.metadata.create_all(bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.post("/api/image/add")
async def add_image(file: UploadFile = File(...), db: Session = Depends(get_db)):
    if not file.filename.lower().endswith(('.png', '.jpg', '.jpeg', '.gif', '.bmp')):
        raise HTTPException(status_code=400, detail="Можно загружать только изображения")

    file_path = os.path.join(UPLOAD_FOLDER, file.filename)
    content = await file.read()

    with open(file_path, "wb") as buffer:
```

Рисунок 1 – программный код

```
49            buffer.write(content)
50
51        with Image.open(file_path) as img:
52            width, height = img.size
53
54        file_size = os.path.getsize(file_path)
55        file_type = file.filename.split('.')[-1].lower()
56
57        db_image = ImageModel(
58            name=file.filename,
59            file_path=file_path,
60            size_bytes=file_size,
61            width=width,
62            height=height,
63            file_type=file_type
64        )
65        db.add(db_image)
66        db.commit()
67        db.refresh(db_image)
68
69        return {
70            "success": True,
71            "message": "Изображение успешно добавлено",
72            "image": {
73                "id": db_image.id,
74                "name": db_image.name,
75                "size_bytes": db_image.size_bytes,
76                "width": db_image.width,
77                "height": db_image.height,
78                "type": db_image.file_type,
79                "created_at": db_image.created_at,
80                "path": db_image.file_path
81            }
82        }
83
84    @app.put("/api/image/change/size")
85    async def change_size(
86        image_path: str = Form(...),
87        new_width: int = Form(...),
88        new_height: int = Form(...),
89        db: Session = Depends(get_db)
90    ):
91        if not os.path.exists(image_path):
92            raise HTTPException(status_code=404, detail="Файл не найден")
93
```

Рисунок 2 – продолжение программного кода

```
94      base, ext = os.path.splitext(image_path)
95      new_path = f"{base}_{new_width}x{new_height}{ext}"
96
97      with Image.open(image_path) as img:
98          resized = img.resize((new_width, new_height))
99          resized.save(new_path)
100
101     with Image.open(new_path) as img:
102         width, height = img.size
103
104     db_image = ImageModel(
105         name=os.path.basename(new_path),
106         file_path=new_path,
107         size_bytes=os.path.getsize(new_path),
108         width=width,
109         height=height,
110         file_type=ext[1:].lower()
111     )
112     db.add(db_image)
113     db.commit()
114
115     return {
116         "success": True,
117         "message": "Размер изображения изменен",
118         "new_file": new_path,
119         "new_size": f"{new_width}x{new_height}"
120     }
121
122 @app.put("/api/image/change/rotate")
123 async def rotate_image(
124     image_path: str = Form(...),
125     degrees: int = Form(90),
126     db: Session = Depends(get_db)
127 ):
128     if not os.path.exists(image_path):
129         raise HTTPException(status_code=404, detail="Файл не найден")
130
131     base, ext = os.path.splitext(image_path)
132     new_path = f"{base}_rotated_{degrees}{ext}"
133
134     with Image.open(image_path) as img:
135         rotated = img.rotate(degrees, expand=True)
136         rotated.save(new_path)
137
138     with Image.open(new_path) as img:
```

Рисунок 3 – продолжение программного кода

```
139         width, height = img.size
140
141     db_image = ImageModel(
142         name=os.path.basename(new_path),
143         file_path=new_path,
144         size_bytes=os.path.getsize(new_path),
145         width=width,
146         height=height,
147         file_type=ext[1:].lower()
148     )
149     db.add(db_image)
150     db.commit()
151
152     return {
153         "success": True,
154         "message": f"Изображение повернуто на {degrees} градусов",
155         "new_file": new_path
156     }
157
158 @app.get("/api/image")
159 async def get_all_images(db: Session = Depends(get_db)):
160     images = db.query(ImageModel).all()
161
162     result = []
163     for img in images:
164         result.append({
165             "id": img.id,
166             "name": img.name,
167             "size_bytes": img.size_bytes,
168             "width": img.width,
169             "height": img.height,
170             "type": img.file_type,
171             "created_at": img.created_at,
172             "file_path": img.file_path
173         })
174
175     return {
176         "total": len(result),
177         "images": result
178     }
179
180 @app.get("/")
181 async def root():
182     return {
183         "message": "Image Processing API - Вариант 1",
184         "methods": [
185             "POST /api/image/add - Добавить изображение",
```

Рисунок 4 – продолжение программного кода

```
186            "PUT /api/image/change/size - Изменить размер",
187            "PUT /api/image/change/rotate - Повернуть изображение",
188            "GET /api/image - Получить все изображения"
189        ],
190        "docs": "/docs"
191    }
```

Рисунок 5 – продолжение программного кода